

Architecture, Design & Build Guide

Portfolio blueprint for a Dynatrace Bluebox Developer Advocate application: architecture, flagship incident, telemetry, AI-agent workflow, UX, testing, documentation, demo strategy, and build plan.

Project in one sentence

Build a deliberately breakable storefront where a developer reproduces a runtime problem, investigates it with Bluebox, gives the evidence to a coding agent, applies a targeted fix, and proves the fix under the exact same workload.

Layer	Recommended component	Purpose
Experience	React + TypeScript	Scenario controls and before/after evidence.
Application	FastAPI / Python	Business logic and intentionally broken endpoint.
Data	PostgreSQL	Deterministic synthetic order data.
Observability	OpenTelemetry	Application and database runtime traces.
Investigation	Bluebox	Runtime debugging context.
Execution	Coding agent	Proposes a patch from evidence; human reviews it.
Reproduction	Python traffic runner	Replays the same workload before and after.
Packaging	Docker Compose	Consistent local setup.

Scope rule: ship one excellent N+1 database-query scenario first. Do not add more incidents until the complete story is reproducible by someone else.

1. What the project must prove

The portfolio signal is not simply that you can build a web app. It is that you can understand a developer tool, create a credible example, teach the concept, anticipate onboarding friction, and help another developer become successful.

Signal	Evidence
Technical credibility	Real API/database behavior, telemetry, deterministic load, tests, and a reviewable fix.
Developer empathy	One-command setup, seed data, prerequisites, verification steps, troubleshooting, useful errors.
Communication	A clear symptom -> evidence -> diagnosis -> change -> proof narrative.
Product understanding	Bluebox is used for runtime investigation rather than recreated as a generic dashboard.
Advocacy judgment	You distinguish Bluebox evidence, coding-agent output, and human review.
Feedback quality	A few concrete onboarding observations with impact and proposed improvements.

Core thesis

Code tells an agent what the application could do. Runtime evidence shows what the application actually did. Make that difference visible.

Use a fictional storefront named **Runtime Mart**. Most customers have a small order history. One seeded customer has hundreds of orders and exposes a deliberately inefficient backend path.

User journey: reset seed -> run healthy baseline -> reproduce slowdown -> inspect runtime telemetry -> pass investigation context to coding agent -> review patch -> replay identical workload -> verify correctness and runtime improvement.

Repository design

```
runtime-rescue-lab/  
apps/web  
apps/api  
scripts/seed.py  
scripts/run_scenario.py  
scripts/verify_fix.py  
tests/integration  
tests/regression  
docs/tutorial.md  
docs/bluebox-review.md  
docs/demo-script.md  
results/example-baseline.json  
results/example-broken.json  
results/example-fixed.json
```

2. Flagship scenario: N+1 database queries

The order-history endpoint first fetches a customer's orders, then performs an additional database query for each order to fetch its items. The response is logically correct, but work grows with the number of orders.

Stage	Behavior
Healthy baseline	Small customer returns quickly and correctness tests pass.
Broken workload	Heavy customer causes many repeated database operations and higher latency.
Investigation	Trace evidence reveals repeated similar database spans.
Patch	Fetch related items in a batch rather than once per order.
Verification	Replay the same seed and workload; response remains correct and repeated DB pattern disappears.

Broken concept

```
orders = get_orders(customer_id)
for order in orders:
    order.items = get_items(order.id) # repeated N times
return orders
```

Target concept

```
orders = get_orders(customer_id)
items = get_items_for_orders(order_ids) # batched
attach_items(orders, items)
return orders
```

Measurement	Broken	Fixed
Correctness	Pass	Pass
DB operation pattern	Grows with order count	Bounded / greatly reduced
Latency	Measurably slower on heavy seed	Materially improved
Trace shape	Repeated similar DB spans	Repeated pattern removed

Benchmark rule

Never promise a specific speedup before measuring it. Publish actual results from your environment and make removal of the pathological query pattern the strongest deterministic proof.

3. Telemetry, Bluebox and the coding agent

Instrument enough to answer the debugging question without turning the project into an observability platform. Create server spans for the order-history request, propagate trace context, and use supported database instrumentation where possible.

Source	Capture
Application telemetry	Route, service, status, errors, stable scenario/dataset identifiers.
Database telemetry	Client spans and repeated operations where supported.
Traffic runner	Wall-clock duration, HTTP status, dataset version.
Verification harness	Correctness hash / assertions, regression status, optional app-owned query counter.

Attribution matters: label every number by its source. Do not show an app-owned query counter and imply Bluebox generated it. Do not treat an AI explanation as proof.

Responsibility split

Actor	Responsibility
Bluebox	Provide runtime investigation context from telemetry.
Coding agent	Use repository + investigation context to propose a code change.
Developer	Review the diff, reject unsafe assumptions, run tests, and verify the result.
Demo harness	Make the experiment repeatable and preserve objective before/after evidence.

Optional evaluation

Run two fresh coding-agent sessions against the same broken commit. Session A gets repository/task context only. Session B also gets runtime investigation context. Keep model, prompt, seed, time budget, and tests constant. Compare correct diagnosis and verified fix rate. Present this as a small demonstration, not a universal benchmark.

Preserve a broken-trace screenshot or recording, the agent diff, result JSON, regression output, and a short limitations note.

4. UI/UX, testing and safety

The custom UI should be a **laboratory control panel**, not a fake observability dashboard.

UI region	What it shows
Header	Project name, environment state, telemetry setup status.
Scenario card	Symptom, dataset, learning objective.
Controls	Reset, baseline, reproduce incident, replay verification.
Results	Duration, status, correctness, app-owned query evidence if available.
Investigation handoff	Instruction to continue the runtime investigation in Bluebox.
Before / after	Same workload, objective measurements, regression status.
Learn panel	What happened, why the fix works, limitations.

Testing matrix

Test	Purpose
Unit	Business rules and batching helpers.
API integration	Response shape, status codes, DB interaction.
Regression	Heavy customer returns identical logical data before and after.
Scenario E2E	Clean seed -> run -> result works.
Performance sanity	Broken build is observably worse than fixed under controlled conditions.
Telemetry smoke	Expected request telemetry is emitted before recording the demo.

Safety

Use synthetic data only. Keep API keys in environment variables. Redact tokens from screenshots. Do not expose public buttons that can generate unbounded load. Protect or localize reset endpoints. Pin dependencies. Do not record sensitive payloads or secrets in telemetry. Store canned example results separately and never silently present them as a live run.

5. Developer Advocate deliverables

Deliverable	Contents
GitHub repository	README, architecture, one-command setup, source, tests, license, troubleshooting.
Written tutorial	Problem -> reproduce -> investigate -> fix -> verify -> limitations.
4-6 minute video	Narrated real workflow; clearly edit waiting time; show evidence and diff.
Onboarding review	Up to four specific friction points: observation, impact, evidence, suggested improvement.
Tester notes	Ask 3-5 developers to follow the guide without coaching; record where they get stuck and what you change.
Example results	Clearly labeled result artifacts from a real run.

README opening you can adapt

Runtime Rescue Lab demonstrates how runtime telemetry can change an AI coding agent's debugging workflow. The lab contains a deliberately seeded performance bug, a reproducible workload, OpenTelemetry instrumentation, and a verification harness. You reproduce the issue, investigate runtime evidence with Bluebox, apply a reviewed patch, and replay the exact workload to verify the result.

15-minute hiring-manager demo

Time	Show
0:00-1:30	Problem, thesis, architecture.
1:30-4:00	Healthy baseline, then reproduce the heavy-account slowdown.
4:00-7:00	Runtime investigation and the evidence that points to repeated DB work.
7:00-10:00	Coding-agent handoff and the proposed diff; explain what you reviewed.
10:00-12:30	Replay the exact workload; show correctness and before/after evidence.
12:30-14:00	Developer-facing tutorial and tester feedback.
14:00-15:00	A few concrete Bluebox onboarding observations and what you would improve.

6. Build plan and definition of done

Day	Target
1	Create repo, Compose stack, schema, deterministic seed, basic endpoint. Validate Bluebox/telemetry path early.
2	Implement deliberate N+1 behavior and scenario runner. Make the slowdown reproducible.
3	Add OpenTelemetry, trace verification, regression tests, and result artifacts.
4	Complete one real Bluebox investigation and coding-agent patch workflow. Refine attribution.
5	Build the small polished UI and before/after view.
6	Write README/tutorial; have at least one other developer attempt setup; fix friction.
7	Record video, prepare onboarding review, rehearse the 15-minute demo, clean repository.

Definition of done

Requirement	Done when
Fresh setup	Another developer can start the project from README instructions.
Reproduction	Heavy scenario fails the performance expectation consistently enough to demonstrate.
Telemetry	You can find the relevant runtime evidence without improvising.
Diagnosis	The evidence clearly supports the N+1 explanation.
Patch	The diff is small, understandable, and reviewable.
Correctness	Regression tests pass before and after.
Verification	Same dataset and workload show the pathological DB pattern is removed.
Teaching	Tutorial and video explain the workflow without requiring private context.
Feedback	Onboarding observations are specific, evidence-based, and actionable.

What NOT to build

Skip authentication, billing, Kubernetes, a custom agent framework, a custom tracing backend, elaborate dashboards, multi-tenant infrastructure, and fancy animation. They add surface area without strengthening the Developer Advocate story.

Best final portfolio package

One excellent incident + working Bluebox integration + reproducible verification + polished README + short narrated video + specific onboarding feedback. That combination demonstrates both engineering and the ability to make other developers successful.